G. H. Raisoni College of Engineering & Management, Wagholi, Pune – 412 207

Department of Master of Computer Applications

D-19

Lab Manual (2025-26)

Pattern-2024

Class: FY Term: II

Python-25MCAPCP205

Faculty Name: Mr.Sachin Zurange

G. H. Raisoni College of Engineering and Management, Wagholi, Pune 412207

Department: MCA

Course Details

Course : Python Lab (25MCAPCP205)

Class: FY MCA **Division:**
Internal Marks: 25 **External marks: 25**
Credits: 2 **Pattern:**

COURSE OUTCOME	
CO1	To Develop Python programs implementing basic concepts and data structures like lists, tuples, dictionaries, and sets.
CO2	To Apply file handling, exception handling, and regular expressions to solve real-world problems.
CO3	To Build Python programs using OOPs concepts like classes, inheritance, constructors, and polymorphism
CO4	To Design interactive GUI applications using Tkinter, and implement web-based CRUD operations using Django framework

List of Experiments

Sr. No.	Experiment List	CO Mapping	Software Required
1	Write a program to create a menu with the following options 1.Addition 2.Subtraction 3.Multiplication 4.Division Accepts users input and perform the operation accordingly. Use functions with arguments.	CO1	Pycharm IDE/Python IDLE/ Anaconda
2	A robot moves in a plane starting from the original point (0,0)(Refer a attached figure). The robot can move toward UP, DOWN, LEFT, RIGHT and STOP with a given steps. The trace of robot movement is shown as the following**(Use loop & if-else concept)** UP 5 DOWN 3 LEFT 3 RIGHT 2 STOP The numbers after the direction are steps. Please write a program to print final position of robot after a sequence of movements. (UP(+)/DOWN(-)=> Y Position, RIGHT(+)/LEFT(-)=> X Position) Accept command from the user in loop till user enters STOP command. **Example: If the following is given as an input to the program:** Enter Command: UP 5 Enter Command: LEFT 3 Enter Command: DOWN 6 Enter Command: RIGHT 4 Enter Command: STOP Then, the output of the program should be: 1, -1	CO1	Pycharm IDE/Python IDLE/ Anaconda
3	Write a python function**(Accept input from User)** 1. To check given number is prime or not. 2. To check given number is spy number or not. **(A number is said to be a Spy number if the sum of all the digits is equal to the product of all digits.** **Eg. 132=> 1*3*2=1+3+2 Input: 132 Output: It is Spy number)**	CO1	Pycharm IDE/Python IDLE/ Anaconda
4	Write a Python program to construct the following pattern, using a nested for loop.**(Use nested for loop** 1. 2 * * * * * * * * * * * *	CO1	Pycharm IDE/Python IDLE/ Anaconda
5	Write a Python program to find palindromes in a given list of strings using Lambda. Orginal list of strings: ['php', 'w3r', 'Python', 'abcd', 'Java', 'aaa'] 1. To Display only List of palindromes o/p ['php', 'aaa'] 2. To Display list in upper case o/p[‘PHP’,’W3R’,’PYTHON’,’JAVA’,AAA’] 3. To Display list in lower case	CO1	Pycharm IDE/Python IDLE/ Anaconda

	o/p[‘php’,’w3r’,’python’,’java’,’aaa’] 4. To Display list in reverse**(Use Slicing Concept)** o/p[‘php’,’r3w’,’nohtyp’,’avaj’,’aaa’]		
6	Consider following list A=[1,4,6,8,7,9,15,18,21] Apply following operation 1. Find out all elements in list lessthen 10 by using list comprehension 2. Find out even numbers list by using filter& anonymous function 3. Find out the maximum & minimum value from the list by using reduce &anonymous function 4. Multiply by 3 number each element of list by using map& anonymous function 5. To append list B[55,66,88] at end of list A & Display final list 6. To Sort List A & Display it.	CO1	Pycharm IDE/Python IDLE/ Anaconda
7	The class has students roll numbered from 1 to 20. **(Use set data type& its operation)** G1 = {1,2,3,4,5,6,7,8,9} G2 = {5,6,7,8,9,10,11,12,13} G3 = {1,2,6,7,11,12,15,16,20} G1 is set of roll numbers who play Cricket. G2 is set of roll numbers who play Football. G3 is set of roll numbers who play Chess. Write a python Script to answer following 1. Find out the roll number of students who play outdoor games only. 2. Find out the roll number of students who play all games. 3. Find out the roll number of students who play nothing. 4. Find out the roll number of students who play "at least one outdoor game" and chess. 5. Find out the roll number of students who play cricket and chess but not football.	CO1	Pycharm IDE/Python IDLE/ Anaconda
8	Write a Python script print a dictionary **(Use Dictionary Data type)** 1. Contains a number (between 1 and n) in the form(x,x*x). Sample Dictionary (n=5) Expected Output: {1:1, 2:4, 3:9, 4:16, 5:25} 2. To display sum all the items in a dictionary. i/p d={1:4,2:5} o/p 4+5=>9 3. Find out the maximum and minimum values from dictionary. 4. To sort a given dictionary by key& display it.	CO1	Pycharm IDE/Python IDLE/ Anaconda
9	Write python script using package/module to calculate area and volume of cube and sphere	CO2	Pycharm IDE/Python IDLE/ Anaconda
10	Write a Python program**(Assume mca.txt file)(Use File Concept)** 1. To read an entire text file& Display it. 2. To read first n lines of a file& Display it. 3. Find the longest words from file &Display it. 4. To count the number of lines, words in a text file& Display it.	CO2	Pycharm IDE/Python IDLE/ Anaconda

11	Write a Python program to accept input number. If number is negative, show the error message "number is negative" otherwise calculate factorial(Use Exception Handling)	CO2	Pycharm IDE/Python IDLE/ Anaconda
12	Write a python program validate PAN card number**(Use regular Expression)** 1. It should be ten characters long. 2. The first five characters should be any upper case alphabets. 3. The next four-characters should be any number from 0 to 9. 4. The last (tenth) character should be any upper case alphabet. 5. It should not contain any white spaces. i/p BNZAA2318J o/p valid pan card number i/p 23ZAABN18J o/p invalid pan card number	CO2	Pycharm IDE/Python IDLE/ Anaconda
13	Write a Python script to Create a Class which Performs Basic Calculator Operations. **(Use Constructor)**	CO3	Pycharm IDE/Python IDLE/ Anaconda
14	Write a python script to define the class person having members name, address. Create a subclass called Employee with member's staffed salary. Create 'n' objects of the Employee class and display all the details of the employee.**(Use Inheritance)**	CO3	Pycharm IDE/Python IDLE/ Anaconda
15	Write Python tkinter GUI program to accept a number n and check whether it is Prime, Perfect or Armstrong number or not. Specify three radio buttons.	CO3	Pycharm IDE/Python IDLE/ Anaconda
16	To implement CRUID operation in Django program	CO4	Pycharm IDE/Python IDLE/ Anaconda
17	To Develop websites & Deploy to Server.	CO4	Pycharm IDE/Python IDLE/ Anaconda

Assignment No:– 1

Aim: Write a program to create a menu with the following options

1.Addition
2.Subtraction
3.Multiplication
4.Division

Accepts users input and perform the operation accordingly. Use functions with arguments.

Theory:

An Introduction to Menu-Driven Program

Menu-Driven Program is a program that gets input from a user by showing the options list, known as the menu, from which the user chooses their option. Systems processing the Menu-Driven programs are ordinary, starting from washing machines controlled by Microprocessors to Automated Teller Machines (ATMs). Taking the ATM case, the user presses single keys to indicate the type of transaction (if the user wants a receipt with the cash, or if an account statement is needed). With many, the user presses a single key to indicate the amount of cash for withdrawal.

The Menu-Driven Systems are beneficial in two ways: At first, input is taken by the single keystrokes, which reduces the chance of the system too prone to user error. Secondly, Menu-Driven Systems limits the characters range resulting in the way where the entered input becomes unambiguous. Hence, these two characteristics make the whole system pretty user-friendly.

Python does not support switch so instead of switch python uses elif ladder to create Menu driven program

The elif statement in Python is short for "else if". It allows you to check multiple conditions and execute different code blocks depending on which condition is true. Here is the syntax for elif:

```
 if condition1:
        # Code block for condition1
elif condition2:
        # Code block for condition2
elif condition3:
        # Code block for condition3
else:
        # Code block if none of the conditions are true
```

Source Code:

```
def addition(a, b):
    return a + b

def subtraction(a, b):
    return a - b

def multiplication(a, b):
    return a * b

def division(a, b):
    if b != 0:
        return a / b
    else:
        return "Division by zero is not allowed"

def menu():
    while True:
        print("\nSelect an operation:")
        print("1. Addition")
        print("2. Subtraction")
        print("3. Multiplication")
        print("4. Division")
        print("5. Exit")

        choice = input("Enter your choice (1-5): ")

        if choice in ['1', '2', '3', '4']:
            try:
                num1 = float(input("Enter the first number: "))
                num2 = float(input("Enter the second number: "))

                if choice == '1':
                    print(f"The result is: {addition(num1, num2)}")
                elif choice == '2':
                    print(f"The result is: {subtraction(num1, num2)}")
                elif choice == '3':
                    print(f"The result is: {multiplication(num1, num2)}")
                elif choice == '4':
                    print(f"The result is: {division(num1, num2)}")
```

```
        except ValueError:
            print("Invalid input! Please enter numeric values.")

    elif choice == '5':
        print("Exiting the program. Goodbye!")
        break
    else:
        print("Invalid choice! Please select a valid option.")

# Run the menu function
menu()
```

Conclusion: Students will able to write menu driven program & user defined function in python

Assignment No:– 2

Aim:

A robot moves in a plane starting from the original point (0,0)(Refer a attached figure). The robot can move toward UP, DOWN, LEFT, RIGHT and STOP with a given steps. The trace of robot movement is shown as the following**(Use loop & if-else concept)**

UP 5 DOWN 3 LEFT 3 RIGHT 2 STOP

The numbers after the direction are steps.

Please write a program to print final position of robot after a sequence of movements. (UP(+)/DOWN(-)=> Y Position, RIGHT(+)/LEFT(-)=> X Position) Accept command from the user in loop till user enters STOP command.

Example: If the following is given as an input to the program:

Enter Command: UP 5
Enter Command: LEFT 3
Enter Command: DOWN 6
Enter Command: RIGHT 4
Enter Command: STOP
Then, the output of the program should be: 1, -1

Theory:

You should use to loop & nested if-else concept

1. The program initializes the robot's position (0, 0).
2. It repeatedly prompts the user to enter a command in the format DIRECTION STEPS.
3. If the user enters STOP, the loop ends.
4. The program checks the command format and updates the x (horizontal) and y (vertical) coordinates based on the direction and steps.
5. Finally, the program prints the robot's final position.

Loop Syntax

1. **While**

Syntax

```
while expression:
   statement(s)
```

e.g

```
count = 0
while (count < 9):
   print 'The count is:', count
   count = count + 1
```

2. **For loop**

Syntax

```
for iterating_var in sequence:
   statements(s)
```

E.g

```
for x in range(6):
  print(x)
```

The range() function returns a sequence of numbers, starting from 0 by default, and increments by 1 (by default), and stops before a specified number.

Syntax

range*(start, stop, step*)

Parameter	Description
Start	Optional. An integer number specifying at which position to start. Default is 0
Stop	Required. An integer number specifying at which position to stop (not included).
Step	Optional. An integer number specifying the incrementation. Default is 1

Source Code:

```
# Initialize the starting position of the robot
x, y = 0, 0

print("Enter commands (e.g., UP 5, DOWN 3). Type STOP to end.")

while True:
    # Get the command from the user
    command = input("Enter Command: ").strip().upper()

    # Check if the command is STOP
    if command == "STOP":
        break
```

```
        # Split the command into direction and steps
        parts = command.split()
        if len(parts) == 2:
            direction, steps = parts[0], parts[1]

            # Validate if steps is a number
            if steps.isdigit():
                steps = int(steps)

                # Update the position based on the direction
                if direction == "UP":
                    y += steps
                elif direction == "DOWN":
                    y -= steps
                elif direction == "LEFT":
                    x -= steps
                elif direction == "RIGHT":
                    x += steps
                else:
                    print("Invalid direction. Please enter UP, DOWN, LEFT, or RIGHT.")
            else:
                print("Invalid steps. Please enter a valid number.")
        else:
            print("Invalid command format. Please enter in 'DIRECTION STEPS' format.")

    # Print the final position
    print(f"Final Position: {x}, {y}")
```

Conclusion: Students will able to use loop &nested if else concept in python

Assignment No:– 3

Aim:

Write a python function**(Accept input from User)**

1. To check given number is prime or not.
2. To check given number is spy number or not.

Theory:

Use loop concept

Prime number

1. The user inputs a number.
2. The function checks if the number is less than or equal to 1 (not prime by definition).
3. It then iterates through numbers from 2 to the square root of the input number (inclusive) to check for divisors.
4. If any divisor is found, the number is not prime.
5. If the loop completes without finding a divisor, the number is prime.

Source Code:

1. **Prime number**

```
# Input from the user
number = int(input("Enter a number: "))

# Prime number check
if number <= 1:
    print(f"{number} is not a prime number.")
else:
    is_prime = True
    for i in range(2, int(number**0.5) + 1):  # Check divisors up to the square root of the number
        if number % i == 0:
            is_prime = False
            break

    if is_prime:
        print(f"{number} is a prime number.")
    else:
        print(f"{number} is not a prime number.")
```

Input:

Enter a number: 17

Output:

17 is a prime number.

2. **Spy number:**

A number is said to be a Spy number if the sum of all the digits is equal to the product of all digits.
Eg. 132=> 1*3*2=1+3+2 Input: 132 Output: It is Spy number)

```
# Input from the user
number = input("Enter a number: ")

# Initialize variables for sum and product of digits
sum_of_digits = 0
product_of_digits = 1

# Iterate through each digit in the number
for digit in number:
    digit = int(digit)  # Convert character to integer
    sum_of_digits += digit
    product_of_digits *= digit

# Check if the number is a spy number
if sum_of_digits == product_of_digits:
    print(f"{number} is a Spy Number.")
else:
    print(f"{number} is not a Spy Number.")
```

Input:

Enter a number: 123

Output:

123 is not a Spy Number.

Conclusion: Students will able to use of various loops concept in python program

Assignment No:– 4

Aim:

Write a Python program to construct the following pattern, using a nested for loop.

(Use nested for loop)

1.

```
*
*   *
*   *   *
```

2.

```
    *
  *   *
*   *   *
```

Theory:

Use next for loop concept

Syntax

```
for i in range(n):
    for j in range(m):
        # Code to execute
```

In this structure:

- i will iterate over range(n) (or any iterable).
- For each value of i, j will iterate over range(m) (or any iterable).

E.g

```
for i in range(3):  # Outer loop
    for j in range(2):  # Inner loop
        print(f"i = {i}, j = {j}")
```

Source Code:

1.

```
*
* *
*   *   *
```

```
# Number of rows in the pattern
rows = 3

for i in range(1, rows + 1):
    for j in range(1, i + 1):
        if i == 3 and j == 2:
            print("    ", end="")  # Add spacing for the third row
        else:
            print("*", end=" ")
    print()  # Move to the next line
```

Output:

2.

```
    *
  *   *
 * *   *
```

```
rows = 3  # Number of rows for the pyramid

# Outer loop for the number of rows
for i in range(rows):
    # Inner loop for printing spaces
    for j in range(rows - i - 1):
        print(" ", end="")

    # Inner loop for printing stars
    for k in range(i + 1):
        print("*", end=" ")

    # Move to the next line after each row
    print()
```

Output:

```
    *
  *   *
 *  *   *
```

Conclusion: Students will able to use of nested for loops concept in python program

Assignment No:– 5

Aim:

Write a Python program following given list using Lambda.
Original list of strings:['php', 'w3r', 'Python', 'abcd', 'Java', 'aaa']

1. To Display only List of palindromes
   o/p ['php', 'aaa']
2. To Display list in upper case
   o/p[‘PHP’,’W3R’,’PYTHON’,’JAVA’,AAA’]
3. To Display list in lower case
   o/p[‘php’,’w3r’,’python’,’java’,’aaa’]
4. To Display list in reverse**(Use Slicing Concept)**
   o/p[‘php’,’r3w’,’nohtyp’,’avaj’,’aaa’]

Theory:

List:

- A list in Python is used to store the sequence of various types of data.
- **Python lists are mutable(Its mean it can modify its element after it created).**
- A list can be defined as a collection of values or items of different types. The items in the list are separated with the comma (,) and enclosed with the square brackets [].
- It allowed duplicate value

Syntax

Listname=[value1, value2,…..valuen]

e.g

list1 = ["apple", "banana", "cherry", "apple", "cherry"]
list2 = ["abc", 34, True, 40, "male"]

Standard Defined List Method

Method	Description	Example
append()	Adds an element at the end of the list	fruits = ['apple', 'banana', 'cherry'] fruits.append("orange") print(x)
copy()	Returns a copy of the list	fruits = ["apple", "banana", "cherry"] x = fruits.copy() print(x)
count()	Returns the number of elements	fruits = ["apple", "banana",

	with the specified value	"cherry"] x = fruits.count("cherry") print(x)
extend()	Add the elements of a list (or any iterable), to the end of the current list	fruits = ['apple', 'banana', 'cherry'] cars = ['Ford', 'BMW', 'Volvo'] fruits.extend(cars) print(fruits)
index()	Returns the index of the first element with the specified value	fruits = ['apple', 'banana', 'cherry'] x = fruits.index("cherry") print(x) o/p 2
reverse()	Reverses the order of the list	fruits = ['apple', 'banana', 'cherry'] fruits.reverse() print(fruits)
sort()	Sorts the list	cars = ['Ford', 'BMW', 'Volvo'] cars.sort() print(cars)

String Standard Defined Method

Method	**Description**
capitalize()	Converts the first character to upper case e.g txt= "hello,andwelcometomyworld." x=txt.capitalize() print (x)
casefold()	Converts string into lower case
center()	Returns a centered string
count()	Returns the number of times a specified value occurs in a string txt= "Iloveapples,applearemyfavoritefruit" x=txt.count("apple") print(x)
encode()	Returns an encoded version of the string
endswith()	Returns true if the string ends with the specified value
expandtabs()	Sets the tab size of the string

find()	Searches the string for a specified value and returns the position of where it was found txt= "Hello,welcometomyworld." x=txt.find("welcome") print(x)
format()	Formats specified values in a string
format_map()	Formats specified values in a string
index()	Searches the string for a specified value and returns the position of where it was found
isalnum()	Returns True if all characters in the string are alphanumeric
isalpha()	Returns True if all characters in the string are in the alphabet
isdecimal()	Returns True if all characters in the string are decimals
isdigit()	Returns True if all characters in the string are digits txt= "50800" x=txt.isdigit() print(x)
isidentifier()	Returns True if the string is an identifier
islower()	Returns True if all characters in the string are lower case
isnumeric()	Returns True if all characters in the string are numeric
isprintable()	Returns True if all characters in the string are printable
isspace()	Returns True if all characters in the string are whitespaces
istitle()	Returns True if the string follows the rules of a title
isupper()	Returns True if all characters in the string are upper case
join()	Joins the elements of an iterable to the end of the string
ljust()	Returns a left justified version of the string
lower()	Converts a string into lower case txt= "HellomyFRIENDS" x=txt.lower() print(x)
lstrip()	Returns a left trim version of the string
maketrans()	Returns a translation table to be used in translations

partition()	Returns a tuple where the string is parted into three parts
replace()	Returns a string where a specified value is replaced with a specified value txt= "Ilikebananas" x=txt.replace("bananas", "apples") print(x)
rfind()	Searches the string for a specified value and returns the last position of where it was found
rindex()	Searches the string for a specified value and returns the last position of where it was found
rjust()	Returns a right justified version of the string
rpartition()	Returns a tuple where the string is parted into three parts
rsplit()	Splits the string at the specified separator, and returns a list
rstrip()	Returns a right trim version of the string
split()	Splits the string at the specified separator, and returns a list txt= "welcometothejungle" x=txt.split() print(x)
splitlines()	Splits the string at line breaks and returns a list
startswith()	Returns true if the string starts with the specified value
strip()	Returns a trimmed version of the string
swapcase()	Swaps cases, lower case becomes upper case and vice versa
title()	Converts the first character of each word to upper case
translate()	Returns a translated string
upper()	Converts a string into upper case txt= "Hellomyfriends" x=txt.upper() print(x)
zfill()	Fills the string with a specified number of 0 values at the beginning

Anonymous Function

- A function without name known as anonymous function.

- In Python anonymous function define by using Lambda function
- A lambda function can take any number of arguments, but can only have one expression.

Syntax

lambda *arguments* : *expression*

E.g

```
x= lambda a:a+ 10
print(x(5))
```

Source Code:

1. Write a Python program following given list using Lambda.
   Original list of strings:['php', 'w3r', 'Python', 'abcd', 'Java', 'aaa']
   - To Display only List of palindromes
     o/p ['php', 'aaa']

```
# Input list
words = ['php', 'w3r', 'Python', 'abcd', 'Java', 'aaa']

# Filter palindromes using a lambda function
palindromes = list(filter(lambda word: word == word[::-1], words))

# Output the result
print(palindromes)
```

Input:

['php', 'w3r', 'Python', 'abcd', 'Java', 'aaa']

Output

['php', 'aaa']

2. Consider following list :['php', 'w3r', 'Python', 'abcd', 'Java', 'aaa'] & convert into upper case using lambda function

Source Code:

```
# Original list
languages = ['php', 'w3r', 'Python', 'abcd', 'Java', 'aaa']

# Convert to uppercase using lambda and map
uppercase_languages = list(map(lambda x: x.upper(), languages))

# Print the result
```

```
print(uppercase_languages)
```

Input:

['php', 'w3r', 'Python', 'abcd', 'Java', 'aaa']

Output:

['PHP', 'W3R', 'PYTHON', 'ABCD', 'JAVA', 'AAA']

3. Consider following list :['php', 'w3r', 'Python', 'abcd', 'Java', 'aaa'] & convert into lower case using lambda function

Source Code:

```
# Original list
my_list = ['php', 'w3r', 'Python', 'abcd', 'Java', 'aaa']

# Convert to lowercase using a lambda function and map
lowercase_list = list(map(lambda x: x.lower(), my_list))

# Print the result
print(lowercase_list)
```

Input:

['php', 'w3r', 'Python', 'abcd', 'Java', 'aaa']

Output:

['php', 'w3r', 'python', 'abcd', 'java', 'aaa']

3.Consider following list :['php', 'w3r', 'Python', 'abcd', 'Java', 'aaa'] & display reverse using lambda function & String Slice Concept

Source Code:

```
# Given list
words = ['php', 'w3r', 'Python', 'abcd', 'Java', 'aaa']

# Reverse each string using lambda and string slicing
reversed_words = list(map(lambda x: x[::-1], words))

print(reversed_words)
```

Input:

['php', 'w3r', 'Python', 'abcd', 'Java', 'aaa']

Output:

['php', 'r3w', 'nohtyP', 'dcba', 'avaJ', 'aaa']

Conclusion: Students will able to use of List & Lambda function concept in python program

Assignment No:– 6

Aim:

Consider following list
A=[1,4,6,8,7,9,15,18,21]
Apply following operation

1. Find out all elements in list less then 10 by using list comprehension
2. Find out even numbers list by using filter& anonymous function
3. Find out the maximum & minimum value from the list by using reduce &anonymous function
4. Multiply by 3 number each element of list by using map& anonymous function
5. To append list B[55,66,88] at end of list A & Display final list
6. To Sort List A & Display it.

Theory:

1. **list comprehension:** List comprehension is a concise way to create lists in Python. It allows you to construct a new list by applying an expression to each element in an iterable, optionally including a Condition.

Syntax:

```
new_list = [expression for item in iterable if condition]
```

- Expression: The operation or value to include in the new list.
- Item: The variable representing each element in the iterable.
- Iterable: The source of elements to iterate over (e.g., a list, range, or string).
- Condition *(optional)*: A filtering condition to include elements that meet a criterion.

E.g

```
squares = [x**2 for x in range(5)]
print(squares) # Output: [0, 1, 4, 9, 16]
```

2. **filter():** The filter() function in Python is used to filter elements from an iterable (like a list, tuple, or string) based on whether they satisfy a given condition (a function that returns True or False).

Syntax:

```
filter(function, iterable)
```

- function: A function that tests each element of the iterable. It should return True for elements to be included in the result and False for elements to be excluded.
- iterable: The iterable (e.g., list, tuple, set) you want to filter.

E.g

```
# Function to check if a number is even
def is_even(num):
return num % 2 == 0

# List of numbers
numbers = [1, 2, 3, 4, 5, 6]

# Using filter
even_numbers = filter(is_even, numbers)

# Converting the result to a list
 print(list(even_numbers)) # Output: [2, 4, 6]
```

3. **map():** The map() function in Python is used to apply a specified function to each item in an iterable (like a list or tuple) and returns a map object (which can be converted to a list, tuple, etc.).

Syntax:

```
map(function, iterable, *iterables)
```

- function: A function that defines the operation to be applied to each item in the iterable.
- iterable: An iterable (e.g., list, tuple, string).
- *iterables: Optional additional iterables. If provided, the function should take as many arguments as there are iterables.

E.g

```
# Function to double a number
def double(x):
   return x * 2

numbers = [1, 2, 3, 4, 5]
result = map(double, numbers)

# Convert map object to list
    print(list(result))  # Output: [2, 4, 6, 8, 10]
```

4. **reduce():** The reduce function in Python is part of the functools module. It applies a specified function cumulatively to the items of an iterable (like a list or tuple) from left to right, reducing the iterable to a single value.

Syntax:

functools.reduce(function, iterable[, initializer])

- function: A function that takes two arguments and returns a single value.
- iterable: The iterable to process.
- initializer (optional): A value that is placed before the items of the iterable in the calculation. If specified, it's used as the initial value.

E.g

```
from functools import reduce

numbers = [1, 2, 3, 4, 5]
result = reduce(lambda x, y: x + y, numbers)
    print(result)  # Output: 15
```

1. Consider following list A=[1,4,6,8,7,9,15,18,21] Find out all elements in list less then 10 by using list comprehension

Source Code:

```
A = [1, 4, 6, 8, 7, 9, 15, 18, 21]
result = [x for x in A if x < 10]
print(result)
```

Input:

A=[1,4,6,8,7,9,15,18,21]

Output:

[1, 4, 6, 8, 7, 9]

2..Find out even numbers list by using filter& anonymous function

Source Code:

```
A = [1, 4, 6, 8, 7, 9, 15, 18, 21]
even_numbers = list(filter(lambda x: x % 2 == 0, A))
print(even_numbers)
```

Input:

A=[1,4,6,8,7,9,15,18,21]

Output:

[4, 6, 8, 18]

3. Find out the maximum & minimum value from the list by using reduce &anonymous function

Source Code:

```
from functools import reduce

# Example list
numbers=[1,4,6,8,7,9,15,18,21]

# Finding maximum value using reduce and lambda
max_value = reduce(lambda x, y: x if x > y else y, numbers)

# Finding minimum value using reduce and lambda
min_value = reduce(lambda x, y: x if x < y else y, numbers)

print(f"Maximum value: {max_value}")
print(f"Minimum value: {min_value}")
```

Input:

numbers=[1,4,6,8,7,9,15,18,21]

Output:

Maximum value:21
Minimum value:1

4. Multiply by 3 number each element of list by using map & anonymous function

Source Code:

```
numbers = [1,4,6,8,7,9,15,18,21]
result = list(map(lambda x: x * 3, numbers))
print(result)
```

Input:
numbers=[1,4,6,8,7,9,15,18,21]

Output:
[3,12,18,24,21,27,45,54,63]

5. To append list B[55,66,88] at end of list A & Display final list

```
A = [1,4,6,8,7,9,15,18,21]
B=[55,66,88]
print(A.extend(B))
```

Input:
A=[1,4,6,8,7,9,15,18,21]

Output:

A=[1,4,6,8,7,9,15,18,21,55,66,88]

6. To Sort List A & Display it

```
A=[1,4,6,8,7,9,15,18,21]
A.sort()
print(A)
```

Input:
A=[1,4,6,8,7,9,15,18,21]

Output:

A=[1,4,6,7,8,9,15,18,21]

Conclusion: Students will able to implement function tools & Lambda function in list

Assignment No:– 7

Aim:

The class has students roll numbered from 1 to 20. **(Use set data type& its operation)**
G1 = {1,2,3,4,5,6,7,8,9}
G2 = {5,6,7,8,9,10,11,12,13}
G3 = {1,2,6,7,11,12,15,16,20}
G1 is set of roll numbers who play Cricket.
G2 is set of roll numbers who play Football.
G3 is set of roll numbers who play Chess.
Write a python Script to answer following
1. Find out the roll number of students who play outdoor games only.
2. Find out the roll number of students who play all games.
3. Find out the roll number of students who play nothing.
4. Find out the roll number of students who play "at least one outdoor game" and chess.
5. Find out the roll number of students who play cricket and chess but not football.

Theory:

- Sets are used to store multiple items in a single variable.
- A set is a collection which is both *unordered* and *unindexed*.
- Sets are written with curly brackets { }.
- It doesn't allowed duplicate value
- It is mutable(The value of the items stored in the set can be changed after set has been created)

Syntax

tupleame={value1, value2,…..valuen}

thisset = {"apple", "banana", "cherry"}

print(thisset)

o/p {'apple', 'cherry', 'banana'}

Set Operation (Union, Intersection, Difference Symmetric)

```
A = {0, 2, 4, 6, 8};
B = {1, 2, 3, 4, 5};

# union
print("Union :", A | B)

# intersection
print("Intersection :", A & B)
```

```
# difference
print("Difference :", A - B)

# symmetric difference means it contain both sets but not in intersection
print("Symmetric difference :", A ^ B)
```

Source Code:

```
# Define the sets
all_students = set(range(1, 21))  # Roll numbers from 1 to 20
G1 = {1, 2, 3, 4, 5, 6, 7, 8, 9}  # Cricket
G2 = {5, 6, 7, 8, 9, 10, 11, 12, 13}  # Football
G3 = {1, 2, 6, 7, 11, 12, 15, 16, 20}  # Chess

# 1. Roll numbers of students who play outdoor games only
outdoor_games_only = (G1 | G2) - G3
print("1. Students who play outdoor games only:", outdoor_games_only)

# 2. Roll numbers of students who play all games
all_games = G1 & G2 & G3
print("2. Students who play all games:", all_games)

# 3. Roll numbers of students who play nothing
play_nothing = all_students - (G1 | G2 | G3)
print("3. Students who play nothing:", play_nothing)

# 4. Roll numbers of students who play "at least one outdoor game" and chess
outdoor_and_chess = (G1 | G2) & G3
print("4. Students who play at least one outdoor game and chess:", outdoor_and_chess)

# 5. Roll numbers of students who play cricket and chess but not football
cricket_chess_not_football = (G1 & G3) - G2
print("5. Students who play cricket and chess but not football:", cricket_chess_not_football)
```

Output:

1. Students who play outdoor games only: {3, 4, 5, 8, 9, 10, 13}
2. Students who play all games: {6, 7}
3. Students who play nothing: {14, 17, 18, 19}
4. Students who play at least one outdoor game and chess: {6, 7, 11, 12}
5. Students who play cricket and chess but not football: {1, 2}

Conclusion: Students will able to implement various set operations in python program

Assignment No:– 8

Aim:

Write a Python script print a dictionary **(Use Dictionary Data type)**
1. Contains a number (between 1 and n) in the form(x,x*x).
Sample Dictionary (n=5)
Expected Output: {1:1, 2:4, 3:9, 4:16, 5:25}
2. To display sum all the items in a dictionary.
i/p d={1:4,2:5}
o/p 4+5=>9
3. Find out the maximum and minimum values from dictionary.
4. To sort a given dictionary by key& display it.

Theory:

- Dictionaries are used to store data values in key:value pairs.
- A dictionary is a collection is unordered,
- It is changeable(mutable)
- It does not allow duplicates.
- Dictionaries are written with curly brackets{}, and have keys and values

Syntax

```
Dictionary name = {
   <key>: <value>,
   <key>: <value>,
     .
     .
     .
   <key>: <value>
}
```

E.g

```
thisdict ={
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}
print(thisdict)
```

o/p

```
{ "brand": "Ford",  "model": "Mustang",  "year": 1964}
```

Methods

Method	Description
clear()	Removes all the elements from the dictionary
copy()	Returns a copy of the dictionary
fromkeys()	Returns a dictionary with the specified keys and value
get()	Returns the value of the specified key
items()	Returns a list containing a tuple for each key value pair
keys()	Returns a list containing the dictionary's keys
pop()	Removes the element with the specified key
popitem()	Removes the last inserted key-value pair
setdefault()	Returns the value of the specified key. If the key does not exist: insert the key, with the specified value
update()	Updates the dictionary with the specified key-value pairs
values()	Returns a list of all the values in the dictionary

Source Code

1. Contains a number (between 1 and n) in the form(x,x*x). Sample Dictionary (n=5)
   Expected Output: {1:1, 2:4, 3:9, 4:16, 5:25}

```
n = 5  # Replace with your desired value of n
result = {x: x * x for x in range(1, n + 1)}
print(result)
```

Output

```
{1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

```
# Input dictionary
d = {1: 4, 2: 5}

# Extract the values and calculate their sum
values = list(d.values())
result = sum(values)

# Display the calculation and result
calculation = ' + '.join(map(str, values))
  print(f"{calculation} => {result}")
```

Input:

```
d={1:4,2:5}
```

Output:

```
9
```

3. Find out the maximum and minimum values from dictionary.

```
# Sample dictionary
data = {'a': 10, 'b': 25, 'c': 5, 'd': 42, 'e': 18}

# Find the maximum and minimum values
max_value = max(data.values())
min_value = min(data.values())

# Display results
print("Maximum value:", max_value)
  print("Minimum value:", min_value)
```

Output:

```
Maximum value: 42
Minimum value: 5
```

4. To sort a given dictionary by key& display it.

```
# Example dictionary
data = {
   'banana': 3,
   'apple': 5,
   'cherry': 2,
   'date': 10
}

# Sorting the dictionary by key
sorted_data = dict(sorted(data.items()))
```

```
# Displaying the sorted dictionary
print("Sorted Dictionary by Key:")
print(sorted_data)
```

Output:

```
Sorted Dictionary by Key:
{'apple': 5, 'banana': 3, 'cherry': 2, 'date': 10}
```

Conclusion: Students will able to implement dictionary & its methods in python program

Assignment No:– 9

Aim:

Write python script using package/module to calculate area and volume of cube and sphere

Theory

Python Modules

A python module can be defined as a python program file which contains a python code including python functions, class, or variables. In other words, we can say that our python code file saved with the extension (.py) is treated as the module

module to be the same as a code library

<u>To creation of User Defined package</u>

1. To create module

```
def show(name):
  print("Hello, " + name)
```

Note: save as mymodule.py

2. The *import* Statement

It can use any Python source file as a module by executing an import statement in some other Python source file.

Syntax

```
import module1[, module2[,... moduleN]
```

```
import mymodule
mymodule.show("om")
```

E.g To add some part in above module

```
person1={
  "name": "om",
  "age": 22,
  "country": "india"
}
```

save above code in mymodule.py

```
import mymodule

a=mymodule.person1["age"]
print(a)
```

Import From Module

It can choose to import only parts from a module, by using the from keyword.

from modname import name1[, name2[, ... nameN]]

```
def show(name):
  print("Hello," +name)


person1={
  "name": "om",
  "age": 22,
  "country": "india"
}
```

Import only the person1 dictionary from the module

```
from mymodule import person1

print (person1["age"])
```

***from...import* * Statement**

It is also possible to import all names from a module into the current namespace by using the following import statement

```
from mymodule import *

print (person1["age"])
show("om")
```

Re-naming a Module

It can create an alias when it import a module, by using the as keyword

import mymodule as mx

```
a=mx.person1["age"]
print(a)
```

Feature	Module	Package	Library
Definition	A single `.py` file	A directory containing modules	A collection of packages and modules
Structure	File with classes, functions, variables	Directory with `__init__.py` and modules	Collection of packages, can contain multiple packages
Purpose	Code organization and reuse	Logical grouping of related modules	Provides high-level functionality
Example	`calculator.py`	`math_tools/`	`NumPy`, `Requests`

Source Code:

Below is a Python script using a custom module to calculate the area and volume of a cube and a sphere. It includes a module file (geometry.py) and a main script (main.py) to utilize the module.

geometry.py
This module contains functions for calculating the area and volume of cubes and spheres.

```
import math

def cube_area(side_length):
    """Calculate the surface area of a cube."""
    if side_length <= 0:
        raise ValueError("Side length must be greater than 0.")
    return 6 * side_length ** 2

def cube_volume(side_length):
    """Calculate the volume of a cube."""
    if side_length <= 0:
        raise ValueError("Side length must be greater than 0.")
    return side_length ** 3

def sphere_area(radius):
    """Calculate the surface area of a sphere."""
    if radius <= 0:
        raise ValueError("Radius must be greater than 0.")
    return 4 * math.pi * radius ** 2

def sphere_volume(radius):
    """Calculate the volume of a sphere."""
    if radius <= 0:
        raise ValueError("Radius must be greater than 0.")
       return (4 / 3) * math.pi * radius ** 3
```

main.py
This script imports the geometry module and uses its functions.

```
import geometry

def main():
    print("Geometry Calculator")
    print("====================")
    try:
        # Cube calculations
        cube_side = float(input("Enter the side length of the cube: "))
        print(f"Cube Surface Area: {geometry.cube_area(cube_side):.2f}")
        print(f"Cube Volume: {geometry.cube_volume(cube_side):.2f}")

        # Sphere calculations
        sphere_radius = float(input("\nEnter the radius of the sphere: "))
        print(f"Sphere Surface Area: {geometry.sphere_area(sphere_radius):.2f}")
        print(f"Sphere Volume: {geometry.sphere_volume(sphere_radius):.2f}")

    except ValueError as e:
        print(f"Error: {e}")

if __name__ == "__main__":
    main()
```

Conclusion: Students will able to implement module, package & library & their differences

Assignment No:– 10

Aim:

Write a Python program**(Assume mca.txt file)(Use File Concept)**
1. To read an entire text file& Display it.
2. To read first n lines of a file& Display it.
3. Find the longest words from file &Display it.
4. To count the number of lines, words in a text file& Display it.

Theory

Function	Description
open()	Used to open a file.
readable()	Used to check whether the file can be read or not.
readline()	Used to read a single line from the file.
readlines()	Used to read all the content in the form of lines.
read()	Used to read whole content at a time.
writable()	Used to check whether the file is writable or not.
write()	Used to write a string to the file.
writelines()	Used to write multiple strings at a time.
close()	Used to close the file from the program.

Source Code

. **1.To read an entire text file& Display it.**

```
# Open the file in read mode
file_path = "mca.txt"

# Read and display the contents of the file
with open(file_path, "r") as file:
    content = file.read()

# Display the content
print("Contents of the file:")
print(content)
```

2. **To read first n lines of a file& Display it.**

```
# Function to read and display the first n lines of a file
def read_first_n_lines(filename, n):
    try:
        with open(filename, 'r') as file:
            for i in range(n):
                line = file.readline()
                if not line:  # Stop if the file has fewer lines than n
                    break
                print(line.strip())
    except Exception as e:
        print(f"An error occurred: {e}")

    # Name of the file
    filename = "mca.txt"
    # Number of lines to read
    n = int(input("Enter the number of lines to read: "))
    # Call the function
    read_first_n_lines(filename, n)
```

3. **Find the longest words from file &Display it.**

```
# Open the file in read mode
    with open("mca.txt", "r") as file:
        # Read all lines from the file
        lines = file.readlines()

    # Initialize an empty list to store all words
    all_words = []

    # Loop through each line in the file
    for line in lines:
        # Split the line into words and add them to the list
        all_words.extend(line.split())

    # Find the length of the longest word
    max_length = max(len(word) for word in all_words)

    # Find all words with the maximum length
    longest_words = [word for word in all_words if len(word) == max_length]

    # Display the longest words
    print("The longest word(s) in the file:")
    for word in longest_words:
            print(word)
```

.4. To count the number of lines, words in a text file& Display it.

```
def count_lines_and_words(file_name):
    # Initialize counters
    line_count = 0
    word_count = 0

    # Open the file in read mode
    with open(file_name, 'r') as file:
        for line in file:
            line_count += 1  # Increment line counter
            word_count += len(line.split())  # Increment word counter by the number of words in the line

    # Display the results
    print(f"Number of lines: {line_count}")
    print(f"Number of words: {word_count}")


    file_name = "mca.txt"
    count_lines_and_words(file_name)
```

Conclusion: Students will able to implement file & its operation in python program

Assignment No:– 11

Aim:

Write a Python program to accept input number. If number is negative, show the error message "number is negative" otherwise calculate factorial(Use Exception Handling)

Theory

The try: block contains one or more statements which are likely to encounter an exception.

If the statements in this block are executed without an exception, the subsequent except: block is skipped.

If the exception does occur, the program flow is transferred to the except: block.

The else keyword to define a block of code to be executed if no errors were raised

Syntax

```
try:
   You do your operations here;
   ......................
except ExceptionI:
   If there is ExceptionI, then execute this block.
except ExceptionII:
   If there is ExceptionII, then execute this block.
   ......................
else:
   If there is no exception then execute this block.
```

e.g

```
try:
    x=int(input('Enter a number: '))
    y=int(input('Enter another number: '))
    z=x/y
except ZeroDivisionError:
       print("Division by 0 not accepted")
else:
    print("else block")
```

Source Code:

```
import math

def main():
    try:
        # Accept user input
        number = int(input("Enter a number: "))

        # Check if the number is negative
        if number < 0:
            raise ValueError("Number is negative")

        # Calculate factorial
        factorial = math.factorial(number)
        print(f"The factorial of {number} is {factorial}")

    except ValueError as e:
        # Handle invalid input or negative number
        print(f"Error: {e}")
    except Exception as e:
        # Handle unexpected errors
        print(f"An unexpected error occurred: {e}")

if __name__== "__main__":
    main()
```

Input & Output

Enter a number: 4
The factorial of 4 is 24

Input & Output

Enter a number: -5
Error: Number is negative

Conclusion: Students will able to handle exception in python program

Assignment No:– 12

Aim:

Write a python program validate PAN card number**(Use regular Expression)**

1. It should be ten characters long.
2. The first five characters should be any upper case alphabets.
3. The next four-characters should be any number from 0 to 9.
4. The last (tenth) character should be any upper case alphabet.
5. It should not contain any white spaces.

i/p BNZAA2318J
o/p valid pan card number
i/p 23ZAABN18J
o/p invalid pan card number

Theory

A RegEx, or Regular Expression, is a sequence of characters that forms a search pattern.

RegEx can be used to check if a string contains the specified search pattern

Special Sequences

A special sequence is a \ followed by one of the characters in the list below, and has a special meaning:

	Character	Description	Example	
	\A	Returns a match if the specified characters are at the beginning of the string	"\AThe"	
	\b	Returns a match where the specified characters are at the beginning or at the end of a word (the "r" in the	r"\bain" r"ain\b"	

	beginning is making sure that the string is being treated as a "raw string")	
\B	Returns a match where the specified characters are present, but NOT at the beginning (or at the end) of a word (the "r" in the beginning is making sure that the string is being treated as a "raw string")	r"\Bain" r"ain\B"
\d	Returns a match where the string contains digits (numbers from 0-9)	"\d"
\D	Returns a match where the string DOES NOT contain digits	"\D"
\s	Returns a match where the string contains a white space character	"\s"
\S	Returns a match where the string DOES NOT contain a white	"\S"

	space character	
\w	Returns a match where the string contains any word characters (characters from a to Z, digits from 0-9, and the underscore _ character)	"\w"
\W	Returns a match where the string DOES NOT contain any word characters	"\W"
\Z	Returns a match if the specified characters are at the end of the string	"Spain\Z"

Sets

A set is a set of characters inside a pair of square brackets [] with a special meaning:

Set	**Description**
[arn]	Returns a match where one of the specified characters (a, r, or n) is present

[a-n]	Returns a match for any lower case character, alphabetically between a and n
[^arn]	Returns a match for any character EXCEPT a, r, and n
[0123]	Returns a match where any of the specified digits (0, 1, 2, or 3) are present
[0-9]	Returns a match for any digit between 0 and 9
[0-5][0-9]	Returns a match for any two-digit numbers from 00 and 59
[a-zA-Z]	Returns a match for any character alphabetically between a and z, lower case OR upper case
[+]	In sets, +, *, ., |, (), $,{} has no special meaning, so [+] means: return a match for any + character in the string

Function	**Description**
findall	Returns a list containing all matches

	E.g import re #Return a list containing every occurrence of "ai": txt = "The rain in Spain" x = re.findall("ai", txt) print(x)

search	Returns a Match object if there is a match anywhere in the string
	
	import re
	
	#Check if the string starts with "The" and ends with "Spain":
	
	txt = "The rain in Spain"
	
	x = re.search("^The.*Spain$", txt)
	
	if x:
	
	  print("YES! We have a match!")
	
	else:
	
	  print("No m*atch")
	
	o/p YES! We have a match

split	Returns a list where the string has been split at each match E.g import re

	#Split the string at every white-space character: txt = "The rain in Spain" x = re.split("\s", txt) print(x)
sub	Replaces one or many matches with a string E.g import re #Replace all white-space characters with the digit "9": txt = "The rain in Spain" x = re.sub("\s", "9", txt) print(x)

Source Code

```
import re

def validate_pan(pan):
    # Define the regular expression for a valid PAN card number
    pattern = r'^[A-Z]{5}[0-9]{4}[A-Z]$'

    # Check if the input matches the pattern
    if re.fullmatch(pattern, pan):
        return "valid PAN card number"
    else:
        return "invalid PAN card number"

# Test cases
print(validate_pan("BNZAA2318J"))  # Output: valid PAN card number
print(validate_pan("23ZAABN18J"))  # Output: invalid PAN card number
```

Output:

BNZAA2318J

valid PAN card number

Conclusion: Students will able to use regular expression in python program

Assignment No:– 13

Aim:

Write a Python script to Create a Class which Performs Basic Calculator Operations. (Use Constructor)

Theory

Creation of class & object

Syntax

class ClassName:

<statement-1>

.

.

<statement-N>

Syntax

Objectname=classname(aguments)

E.g 1

```
class Person:
  def _init_(self):#constrcutor
    self.name="amol"
    self.age = 36


  def myfunc(self):
    print("name is " + self.name)
    print("age is " + self.age)


p1=Person()#instance_creation

print(p1.name)
print(p1.age)
```

Constructor

- A constructor is a special type of method (function) that is called automatically whenever an object of that class is created.
- A constructor is a special type of method (function) which is used to initialize the instance members of the class.
- The name of constructor is always name _init()_
- Constructors can be of non-parametrized & parameterized.

E.g
1. non-parametrized

```
class Person:
  def _init_(self)#constrcutor
    self.name="amol"
    self.age = 36


p1 = Person()
```

E.g
2 parameterized

```
class Person:
  def _init_(self,name,age):#constrcutor
    self.name=name
    self.age = age


  p1 = Person("amol", 36)
```

Source code

```
class Calculator:
    def _init_(self, num1, num2):
        """
        Constructor to initialize two numbers.
        :param num1: First number
        :param num2: Second number
        """
        self.num1 = num1
        self.num2 = num2
```

```
    def add(self):
        """Returns the sum of num1 and num2."""
        return self.num1 + self.num2

    def subtract(self):
        """Returns the difference between num1 and num2."""
        return self.num1 - self.num2

    def multiply(self):
        """Returns the product of num1 and num2."""
        return self.num1 * self.num2

    def divide(self):
        """
        Returns the quotient of num1 divided by num2.
        Raises:
            ZeroDivisionError: If num2 is zero.
        """
        if self.num2 == 0:
            raise ZeroDivisionError("Division by zero is not allowed.")
        return self.num1 / self.num2

# Example usage
if _name___== "_main_":
    try:
        # Initialize the calculator with two numbers
        num1 = float(input("Enter the first number: "))
        num2 = float(input("Enter the second number: "))
        calc = Calculator(num1, num2)

        # Perform operations
        print(f"Addition: {calc.add()}")
        print(f"Subtraction: {calc.subtract()}")
        print(f"Multiplication: {calc.multiply()}")
        print(f"Division: {calc.divide()}")

    except ZeroDivisionError as e:
        print(e)
    except ValueError:
print("Invalid input! Please enter numeric values.")
```

Conclusion: Students will able to use object oriented concept in python program

Assignment No:– 14

Aim:

Write a python script to define the class person having members name, address. Create a subclass called Employee with member's staffed salary. Create 'n' objects of the Employee class and display all the details of the employee.(Use Inheritance)

Theory

Python Inheritance

Inheritance allows us to define a class that inherits all the methods and properties from another class.

Parent class is the class being inherited from, also called base class.

Child class is the class that inherits from another class, also called derived class.

Type of inheritance

1. **Single Inheritance:** A derived class is **inherited** from the only one base class known as Single inheritance .

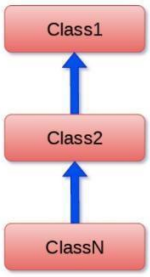
Syntax

```
class derived-class(base class):
    <class-suite>
```

E.g

```
class A:
    def disp1(self):
        print("disp1 method in base")
class B(A):
    def disp2(self):
        print("disp2 method in derived")
ob = B()
ob.disp1()
ob.disp2()
```

2. **MultiLevel Inheritance:** Multi-level inheritance is archived when a derived class inherits another derived class.



Syntax

```
class class1:
    <class-suite>
class class2(class1):
    <class suite>
class class3(class2):
    <class suite>
```

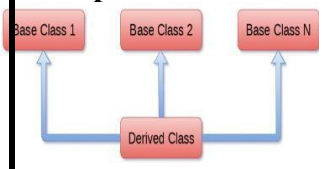
e.g

```
class A:
    def disp1(self):
        print("disp1 method in base")
class B(A):
    def disp2(self):
        print("disp2 method in derived1")

class C(B):
    def disp3(self):
        print("disp3 method in derived2")

ob = C()
ob.disp1()
ob.disp2()
ob.disp3()
```

3. **Multiple inheritance:** To inherit multiple base classes in the child class known as multiple inheritance.



Syntax

```
class Base1:
    <class-suite>
```

```
class Base2:
   <class-suite>

class BaseN:
  <class-suite>

class Derived(Base1, Base2, ..... BaseN):
   <class-suite>
```

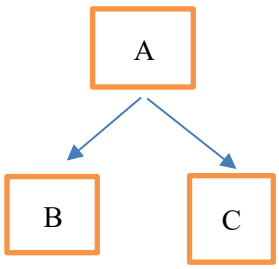
```
e.g
class A:
   def disp1(self):
      print("disp1 method in base")
class B:
   def disp2(self):
      print("disp2 method in derived1")

class C(A,B):
   def disp3(self):
      print("disp3 method in derived2")

ob = C()
ob.disp1()
ob.disp2()
ob.disp3()
```

4. Hierarchical Inheritance

Hierarchical inheritance involves more then derived classes derived from the single base or parent class.



```
class A:
    def disp1(self):
        print("disp1 method in base")
class B(A):
    def disp2(self):
        print("disp2 method in derived1")

class C(A):
    def disp3(self):
        print("disp3 method in derived2")

ob1 = B()
ob1.disp1()
ob1.disp2()

ob2=C()
ob2.disp1()
ob2.disp3()
```

Source Code:

```
class Person:
    """Class to represent a person with name and address."""
    def _init_(self, name, address):
        self.name = name
        self.address = address

    def display_details(self):
        """Displays the name and address of the person."""
        print(f"Name: {self.name}")
        print(f"Address: {self.address}")

class Employee(Person):
    """Subclass of Person representing an employee with additional attributes."""
    def _init_(self, name, address, salary):
        super()._init_(name, address)
        self.salary = salary

    def display_details(self):
        """Displays all details of the employee."""
        super().display_details()
        print(f"Salary: {self.salary}")
```

```
# Function to create and display details of 'n' employees
def main():
    employees = []
    n = int(input("Enter the number of employees: "))

    for i in range(n):
        print(f"\nEnter details for Employee {i + 1}:")
        name = input("Enter name: ")
        address = input("Enter address: ")
        salary = float(input("Enter salary: "))
        employees.append(Employee(name, address, salary))

    print("\nEmployee Details:")
    for i, employee in enumerate(employees, start=1):
        print(f"\nEmployee {i}:")
        employee.display_details()

if __name__== "__main__":
main()
```

Conclusion: Students will able to use inheritance type in python program

Assignment No:– 15

Aim:

Write Python tkinter GUI program to accept a number n and check whether it is Prime, Perfect or Armstrong number or not. Specify three radio buttons

Theory

Tkinter(GUI) Programming

Python provides the standard library Tkinter for creating the graphical user interface for desktop based applications.

- **Tkinter top-level window can be created by using the following steps.**

1. import the Tkinter module.
2. Create the main application window.
3. Add the widgets like labels, buttons, frames, etc. to the window.
4. Call the main event loop so that the actions can take place on the user's computer screen.

Source Code

```
import tkinter as tk
from tkinter import messagebox

def is_prime(n):
    if n < 2:
        return False
    for i in range(2, int(n ** 0.5) + 1):
        if n % i == 0:
            return False
    return True

def is_perfect(n):
    if n < 1:
        return False
    return sum(i for i in range(1, n) if n % i == 0) == n

def is_armstrong(n):
    digits = list(map(int, str(n)))
    return n == sum(d ** len(digits) for d in digits)
```

```
def check_number():
try:
    n = int(entry.get())
    if n < 0:
        messagebox.showerror("Error", "Please enter a positive integer.")
        return

    selected_option = selected_check.get()
    if selected_option == "prime":
        result = is_prime(n)
        result_text = "a Prime" if result else "not a Prime"
    elif selected_option == "perfect":
        result = is_perfect(n)
        result_text = "a Perfect" if result else "not a Perfect"
    elif selected_option == "armstrong":
        result = is_armstrong(n)
        result_text = "an Armstrong" if result else "not an Armstrong"
    else:
        messagebox.showerror("Error", "Please select an option.")
        return

    messagebox.showinfo("Result", f"The number {n} is {result_text} number.")
except ValueError:
    messagebox.showerror("Error", "Please enter a valid integer.")

# GUI setup
root = tk.Tk()
root.title("Number Checker")

# Input field
frame = tk.Frame(root)
frame.pack(pady=10)

tk.Label(frame, text="Enter a number:").grid(row=0, column=0, padx=5, pady=5)
entry = tk.Entry(frame, width=15)
entry.grid(row=0, column=1, padx=5, pady=5)

# Radio buttons
selected_check = tk.StringVar(value="")
radiobutton_frame = tk.Frame(root)
radiobutton_frame.pack(pady=10)

tk.Label(radiobutton_frame, text="Check if the number is:").pack(anchor="w")

radio_prime = tk.Radiobutton(radiobutton_frame, text="Prime", variable=selected_check, value="prime")
radio_prime.pack(anchor="w")

radio_perfect = tk.Radiobutton(radiobutton_frame, text="Perfect", variable=selected_check, value="perfect")
radio_perfect.pack(anchor="w")

radio_armstrong = tk.Radiobutton(radiobutton_frame, text="Armstrong", variable=selected_check, value="armstrong")
radio_armstrong.pack(anchor="w")
```

```
# Check button
check_button = tk.Button(root, text="Check", command=check_number)
check_button.pack(pady=10)

# Start the Tkinter main loop
root.mainloop()
```

Conclusion: Students will able to use tkinter & its widgets in python program

Assignment No:– 16

Aim:

To implement CRUID operation in Django program

Theory

The MVT (Model View Template) is a software design pattern. It is a collection of three important

components Model View and Template. The Model helps to handle database. It is a data access layer which handles the data.

The Template is a presentation layer which handles User Interface part completely. The View is

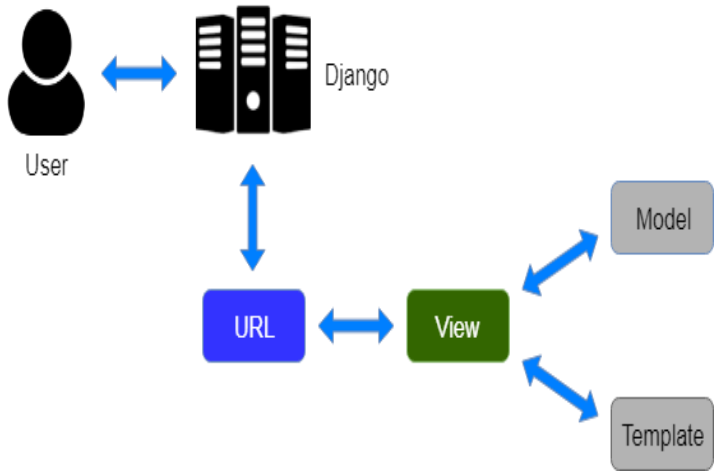
used to execute the business logic and interact with a model to carry data and renders a template.

Although Django follows MVC pattern but maintains it?s own conventions. So, control is handled by the framework itself.

There is no separate controller and complete application is based on Model View and Template.

That?s why it is called MVT application.

See the following graph that shows the MVT based control flow.



Here, a user **requests** for a resource to the Django, Django works as a controller and

check to the available resource in URL.

If URL maps, **a view is called** that interact with model and template, it renders a template.

Django responds back to the user and sends a template as a **response**.

Source Code

1. Set Up Django Project
Create a new Django project and app:

```
django-admin startproject myproject
cd myproject
python manage.py startapp myapp
```

2. Install and Configure MySQL
Install the mysqlclient package:

```
pip install mysqlclient
```

Update the DATABASES configuration in settings.py to use MySQL:

```
# myproject/settings.py
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.mysql',
        'NAME': 'your_database_name',
        'USER': 'your_username',
        'PASSWORD': 'your_password',
        'HOST': 'localhost',  # or your database host
        'PORT': '3306',       # MySQL default port
    }
}
```

3. Define a Model
Create a model to represent the data:

```
# myapp/models.py
from django.db import models
```

```
class Item(models.Model):
    name = models.CharField(max_length=100)
    description = models.TextField()
    price = models.DecimalField(max_digits=10, decimal_places=2)
created_at = models.DateTimeField(auto_now_add=True)
updated_at = models.DateTimeField(auto_now=True)

def __str__(self):
    return self.name
```

4. Create Views for CRUD Operations

Define views in views.py for each operation:

```
# myapp/views.py
from django.shortcuts import render, get_object_or_404, redirect
from .models import Item
from .forms import ItemForm

def item_list(request):
    items = Item.objects.all()
    return render(request, 'myapp/item_list.html', {'items': items})

def item_create(request):
    if request.method == "POST":
        form = ItemForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('item_list')
    else:
        form = ItemForm()
    return render(request, 'myapp/item_form.html', {'form': form})

def item_update(request, pk):
    item = get_object_or_404(Item, pk=pk)
    if request.method == "POST":
        form = ItemForm(request.POST, instance=item)
        if form.is_valid():
            form.save()
            return redirect('item_list')
    else:
        form = ItemForm(instance=item)
    return render(request, 'myapp/item_form.html', {'form': form})

def item_delete(request, pk):
    item = get_object_or_404(Item, pk=pk)
    if request.method == "POST":
        item.delete()
        return redirect('item_list')
    return render(request, 'myapp/item_confirm_delete.html', {'item': item})
```

5. Create Forms

Use Django forms for input validation:

```
# myapp/forms.py
from django import forms
from .models import Item

class ItemForm(forms.ModelForm):
    class Meta:
        model = Item
        fields = ['name', 'description', 'price']
```

6. Create Templates
Create HTML templates for each view:

```
<h1>Items</h1>
<a href="{% url 'item_create' %}">Add Item</a>
<ul>
    {% for item in items %}
        <li>
            {{ item.name }} - {{ item.price }}
            <a href="{% url 'item_update' item.id %}">Edit</a>
            <a href="{% url 'item_delete' item.id %}">Delete</a>
        </li>
    {% endfor %}
</ul>
```

item_form.html

```
<h1>{{ form.instance.id|yesno:"Edit Item,Add Item" }}</h1>
<form method="post">
    {% csrf_token %}
    {{ form.as_p }}
    <button type="submit">Save</button>
</form>
```

item_confirm_delete.html

```
<h1>Delete Item</h1>
<p>Are you sure you want to delete "{{ item.name }}"?</p>
<form method="post">
    {% csrf_token %}
    <button type="submit">Confirm</button>
</form>
<a href="{% url 'item_list' %}">Cancel</a>
```

7. Configure URLs
Add URL patterns in urls.py:

```
# myapp/urls.py
from django.urls import path
from . import views

urlpatterns = [
path('', views.item_list, name='item_list'),
path('create/', views.item_create, name='item_create'),
path('update/<int:pk>/', views.item_update, name='item_update'),
path('delete/<int:pk>/', views.item_delete, name='item_delete'),
]
```

Include the app's URLs in the project's urls.py:

```
# myproject/urls.py
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('myapp.urls')),
]
```

8. Run the Server

Start the development server:

```
python manage.py runserver
```

Conclusion: Students will able to use Django CRUD operation in python program

Assignment No:– 17

Aim:

To Develop websites & Deploy to Server.

Theory

General Steps for Django Deployment

1. Prepare Your Django Project

- Ensure your project is ready for deployment:
  - Set DEBUG = False in settings.py.
  - Add the allowed host(s) to the ALLOWED_HOSTS list in settings.py (e.g., ALLOWED_HOSTS = ['yourdomain.com', '127.0.0.1']).
  - **Collect static files by running:**
    python manage.py collectstatic

Create a **requirements.txt** file to list dependencies

pip freeze > requirements.txt

Ensure you have a **Procfile** for platforms like Render or Railway

web: gunicorn your_project_name.wsgi

2. Choose a Free Hosting Platform

Here are some popular free hosting options for Django:

Option 1: Deploy on Render

1. **Create an Account**: Sign up at Render.
2. **Push Code to Git Repository**: Ensure your project is hosted on GitHub, GitLab, or Bitbucket.
3. **Create a New Web Service**:
   - Go to the Render dashboard and click **New > Web Service**.
   - Connect your Git repository and select the branch to deploy.
4. **Configure Build Settings**:
   - Add a build command: pip install -r requirements.txt.
   - Use the **Gunicorn** command for the start script

gunicorn your_project_name.wsgi --bind 0.0.0.0:$PORT

Set Environment Variables:

- Add DJANGO_SETTINGS_MODULE and other necessary environment variables like SECRET_KEY.

Deploy:
- Render will automatically deploy your app.

Option 3: Deploy on PythonAnywhere

1. **Create an Account**: Sign up at PythonAnywhere.
2. **Upload Your Files**:
   - Use the web interface or scp to upload your Django project.
3. **Set Up a Virtual Environment**:
   - Create a virtual environment and install dependencies:

```
mkvirtualenv myenv --python=python3
pip install -r requirements.txt
```

4. **Configure the Web App**:
   - Go to the **Web** tab and create a new web app.
   - Select Django and the Python version.
   - Configure the WSGI file to point to your wsgi.py.
5. **Set Static Files**:
   - Update the static files directory in the **Static Files** section.
6. **Run Migrations**

Run database migrations in the console:

```
python manage.py migrate
```

Notes
- Free hosting platforms may have limits on traffic or runtime.
- Use cloud databases (e.g., PostgreSQL on Render or Railway) if your hosting service doesn't support databases.

Conclusion: Students will able to deploy Django in python program